

Lec 9+10

AUTO-ASSOCIATIVE NETS

Dr. Mohammad Mhawish

- An associative memory network can **store** a set of patterns as **memories**. When an associative memory is being presented with a key pattern, it responds by producing one of the stored patterns, which closely resembles or relates to the key pattern.
- Thus, the recall is through association of the key pattern, with the help of information memorized.
- These types of memories are also called as *content-addressable memories* (CAM) in contrast to that of traditional *address-addressable memories* in digital computers where stored pattern (in bytes) is recalled by its address.

- Associative memory makes a parallel search within a stored data file. The concept behind this search is to output any one or all stored items which match the given search argument and to retrieve the stored data either completely or partially.

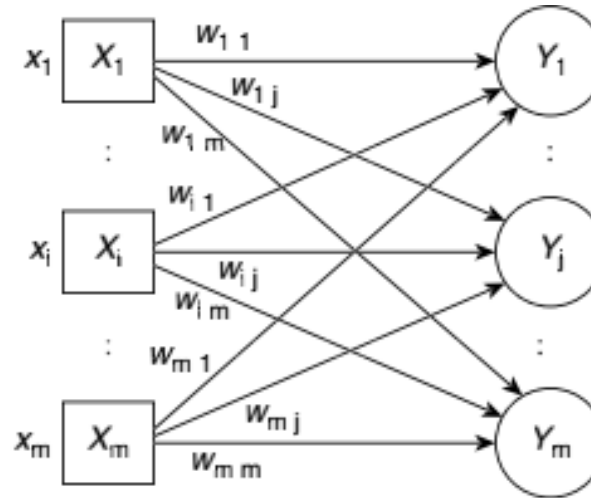
Two types of associative memories can be differentiated. They are ***autoassociative*** memory and ***heteroassociative*** memory.

- Both these nets are single-layer nets in which the weights are determined in a manner that the net stores a set of pattern associations. Each of this association is an input–output vector pair, say, $s:t$. If each of the output vectors is same as the input vectors with which it is associated, then the net is said to be autoassociative memory net. On the other hand, if the output vectors are different from the input vectors then the net is said to be heteroassociative memory net.

- Regarding architecture, an associative memory neural net may be either *feedforward* type or *recurrent (iterative)* type.
- In feedforward nets, signals are unidirectional. They flow from input to output and not in the reverse direction.
- In recurrent (or iterative) nets, signals flow back and forth between the input and the output until the net reaches an equilibrium state.

AUTO-ASSOCIATIVE NETS

- The input and output patterns of an auto-associative net are the same.
- Presented with an input pattern, perhaps noisy to some extent, an auto-associative net returns the same pattern (this time without any noise) in case the input matches one of the stored patterns. The architecture of an auto-associative net is shown in Fig. It has the same number of input and output units.



Training

- An auto-associative network can be trained with Hebb, delta, Outer Products Rule. However, for the sake of simplicity, here we apply only the Hebb rule for training an associative net. Let us first consider the case of storing a single pattern $\mathbf{s} = [s_1, s_2, \dots, s_m]$ on an auto-associative net. The weight matrix W of the net is obtained as

$$W = \mathbf{s}^T \times \mathbf{s} = \begin{pmatrix} s_1 \\ \vdots \\ s_i \\ \vdots \\ s_m \end{pmatrix} \times (s_1 \cdots s_j \cdots s_m) = \begin{pmatrix} s_1 \times s_1 & \cdots & s_1 \times s_j & \cdots & s_1 \times s_m \\ \vdots & & & & \\ s_i \times s_1 & \cdots & s_i \times s_j & \cdots & s_i \times s_m \\ \vdots & & & & \\ s_m \times s_1 & \cdots & s_m \times s_j & \cdots & s_m \times s_m \end{pmatrix}$$

Let us find the weight matrix of an auto-associative net to store and recognize the vector $\mathbf{s} = [1, -1, -1, 1]$. Applying Equation 8.1 we get the following weight matrix.

$$W = \mathbf{s}^T \times \mathbf{s} = \begin{pmatrix} 1 \\ -1 \\ -1 \\ 1 \end{pmatrix} \times (1 \ -1 \ -1 \ 1) = \begin{pmatrix} 1 & -1 & -1 & 1 \\ -1 & 1 & 1 & -1 \\ -1 & 1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}$$

Application

Once the weight matrix is obtained through the appropriate training method, it is ready for application. Let $\mathbf{x} = [x_1, \dots, x_i, \dots, x_m]$ be an arbitrary input pattern of length m . To recognize the input pattern $\mathbf{x}$, we have to determine whether the given pattern matches with the stored pattern. The following steps are carried out to achieve this.

1. For each output unit $Y_j, j = 1, \dots, m$, compute the net input y_in_j to Y_j .

$$y_in_j = \sum_{i=1}^m x_i \cdot w_{ij}$$

In matrix notation, $\mathbf{Y_in} = \mathbf{x} \times \mathbf{W}$, where $\mathbf{Y_in} = [y_in_1, \dots, y_in_m]$, $\mathbf{x} = [x_1, \dots, x_m]$, and $\mathbf{W}$ is the weight matrix.

2. For each output unit $Y_j, j = 1, \dots, m$, find the activation using the function

$$y_out = \begin{cases} 1, & \text{if } y_in > 0, \\ -1, & \text{otherwise} \end{cases}$$

If the output vector thus found is identical to the stored pattern, i.e., $\mathbf{Y_out} = [y_out_1, \dots, y_out_m] = [s_1, \dots, s_m] = \mathbf{s}$ then the input is recognized, otherwise not.

- Refer to the auto-associative net built in above Example . To see if the resultant net can recognize the stored pattern $x = [1, -1, -1, 1]$, we follow the steps described above.

1. Compute the vector Y_{in} for net inputs to the output units.

$$\begin{aligned} Y_{in} &= [y_{in_1}, y_{in_2}, y_{in_3}, y_{in_4}] \\ &= [1 \ -1 \ -1 \ 1] \times \begin{pmatrix} 1 & -1 & -1 & 1 \\ -1 & 1 & 1 & -1 \\ -1 & 1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix} \\ &= [4, -4, -4, 4] \end{aligned}$$

2. Find the output vector employing the activation function.

$$\begin{aligned} Y_{out} &= [y_{out_1}, y_{out_2}, y_{out_3}, y_{out_4}] \\ &= [1, -1, -1, 1] \end{aligned}$$

Hence, the net has acquired the ability to recognize the stored pattern.

Elimination of Self-connection

- It is convenient to set the diagonal elements of the weight matrix W to 0 to indicate that the net does not have any self-connection. This is more useful for nets that store several patterns. For such nets, the diagonal elements are set to 0s to ensure that these elements do not dominate during application of the net. Otherwise, the net has a tendency to reproduce the input pattern rather than the stored pattern.

- The weight matrix of the net cited in above Example becomes, with all diagonal elements set to 0

$$W = \begin{bmatrix} 0 & -1 & -1 & 1 \\ -1 & 0 & 1 & -1 \\ -1 & 1 & 0 & -1 \\ 1 & -1 & -1 & 0 \end{bmatrix}$$

Therefore, the net input to the output units are computed as

$$Y_{in} = [y_{in_1}, y_{in_2}, y_{in_3}, y_{in_4}]$$

$$\begin{aligned} &= [1 \ -1 \ -1 \ 1] \times \begin{pmatrix} 0 & -1 & -1 & 1 \\ -1 & 0 & 1 & -1 \\ -1 & 1 & 0 & -1 \\ 1 & -1 & -1 & 0 \end{pmatrix} \\ &= [3, -3, -3, 3] \end{aligned}$$

And the output vector is obtained as

$$\begin{aligned} Y_{out} &= [y_{out_1}, y_{out_2}, y_{out_3}, y_{out_4}] \\ &= [1, -1, -1, 1] \end{aligned}$$

Hence, the net retains the ability to recognize the stored pattern.

Recognition of Noisy Patterns

- Mere recognition of the pattern already stored in the net is not sufficient because this could as well be achieved with a diagonal weight matrix. The strength of a neural net lies in its tolerance of noisy input pattern, provided the input is sufficiently close to the stored pattern in spite of it being noisy. Two kinds of noises may appear in the input pattern. These are (a) missing elements and (b) erroneous elements. A missing element is represented by 0, instead of a 1 or -1, in the appropriate place. An erroneous element presents the complement of the correct value, i.e., 1 (-1) in place of -1 (1). The following example illustrates the capacity of auto-associative nets to recognize noisy patterns.

Let us consider a noisy input pattern where the leftmost element of the input vector is missing. Thus, the input pattern appears as $[0 \ -1 \ -1 \ 1]$ instead of $[1 \ -1 \ -1 \ 1]$. Computation of the net input and subsequently the output vector is done in the usual way.

$$Y_{in} = [y_{in_1}, y_{in_2}, y_{in_3}, y_{in_4}]$$

$$= [0 \ -1 \ -1 \ 1] \times \begin{pmatrix} 0 & -1 & -1 & 1 \\ -1 & 0 & 1 & -1 \\ -1 & 1 & 0 & -1 \\ 1 & -1 & -1 & 0 \end{pmatrix}$$

$$= [3, -2, -2, 2]$$

$$Y_{out} = [y_{out_1}, y_{out_2}, y_{out_3}, y_{out_4}]$$

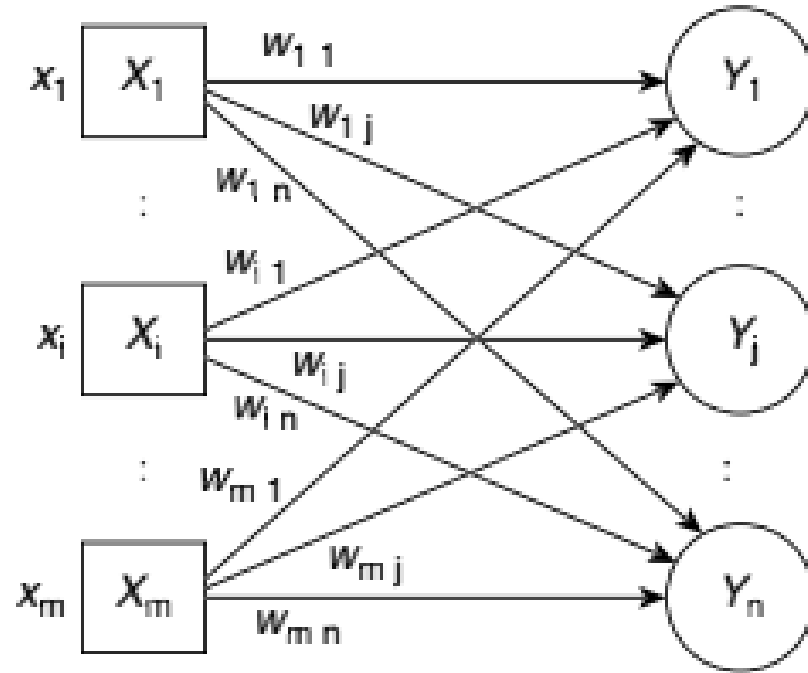
$$= [1, -1, -1, 1]$$

#	Input pattern (noisy)	Net input to the output layer ($Y_{in} = s \times W$)	Output pattern
i	[1, 0, -1, 1]	[2, -3, -2, 2]	[1, -1, -1, 1]
ii	[1, -1, 0, 1]	[2, -2, -3, 2]	[1, -1, -1, 1]
iii	[1, -1, -1, 0]	[2, -2, -2, 3]	[1, -1, -1, 1]

HETERO-ASSOCIATIVE NETS

- An auto-associative net stores the association between identical patterns so that given an input pattern, it can recall whether it is one of the stored patterns or not. In contrast, a hetero-associative net is used to store the associations between pairs of patterns that are not identical. This means, if $\mathbf{s} : \mathbf{t}$ be a pair of associated patterns and this association is stored in a hetero-associative net, then $\mathbf{s} \neq \mathbf{t}$.

- The structure of a hetero-associative net is shown in Fig. 8.2. It is almost identical to the structure of an auto-associative net depicted in Fig except that in case of hetero-associative networks the number of input units (m) is not necessarily equal to the number of the output units (n).



- Let $\mathbf{s} : \mathbf{t}$ be an association of two patterns where $\mathbf{s} = [s_1, s_2, \dots, s_m]$ and $\mathbf{t} = [t_1, t_2, \dots, t_n]$. Let W be the weight matrix of the hetero-associative net storing the association $s : t$.

$$W = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ \vdots & \ddots & & \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix}$$

Then W is obtained using the Hebb rule as described below.

$$w_{ij} = s_i \times t_j \quad (i = 1, \dots, m \text{ and } j = 1, \dots, n)$$

Using matrix notation, the relation becomes

$$W = \mathbf{s}^T \times \mathbf{t} = \begin{pmatrix} s_1 \\ \vdots \\ s_i \\ \vdots \\ s_m \end{pmatrix} \times (t_1 \cdots t_j \cdots t_n) = \begin{pmatrix} s_1 \times t_1 & \cdots & s_1 \times t_j & \cdots & s_1 \times t_n \\ \vdots & & \vdots & & \vdots \\ s_i \times t_1 & \cdots & s_i \times t_j & \cdots & s_i \times t_n \\ \vdots & & \vdots & & \vdots \\ s_m \times t_1 & \cdots & s_m \times t_j & \cdots & s_m \times t_n \end{pmatrix}$$

Consider a pair $\mathbf{s} : \mathbf{t}$ of patterns where $\mathbf{s} = [1, -1, 1, -1]$ and $\mathbf{t} = [-1, 1, -1]$. The weight matrix of the net storing this association is calculated below.

$$W = \mathbf{s}^T \times \mathbf{t} = \begin{pmatrix} 1 \\ -1 \\ 1 \\ -1 \end{pmatrix} \times (-1 \ 1 \ -1) = \begin{pmatrix} -1 & 1 & -1 \\ 1 & -1 & 1 \\ -1 & 1 & -1 \\ 1 & -1 & 1 \end{pmatrix}$$

- *(Application of hetero-associative net for single association)*

To verify if the net developed in previous is able to behave in the expected manner, we need to find the output vector for the input pattern $\mathbf{x} = [1, -1, 1, -1]$.

- Input pattern: $\mathbf{x} = [1, -1, 1, -1]$

Net input to the output layer: $\mathbf{Y}_{in} = \mathbf{x} \times W = [1, -1, 1, -1] \times W = [-4, 4, -4]$

Activation at the output layer: $\mathbf{Y}_{out} = [-1, 1, -1] = \mathbf{s}$

Hence the net associates the input $[1, -1, 1, -1]$ to the stored pattern $[-1, 1, -1]$.

- *(Recognition of noisy input by hetero-associative net)*

Consider again the net developed in previous example Suppose the input to the net is $\mathbf{x} = [1, -1, 0, -1]$

where the third element is missing. Performance of the net for such an input is described below.

- Input pattern: $\mathbf{x} = [1, -1, 0, -1]$

Net input to the output layer: $\mathbf{Y}_{in} = \mathbf{x} \times W = [1, -1, 0, -1] \times W = [-3, 3, -3]$

Activation at the output layer: $\mathbf{Y}_{out} = [-1, 1, -1] = \mathbf{s}$

Hence, the net correctly associates the noisy input pattern $[1, -1, 0, -1]$ to the stored pattern $[-1, 1, -1]$.

HOPFIELD NETWORKS

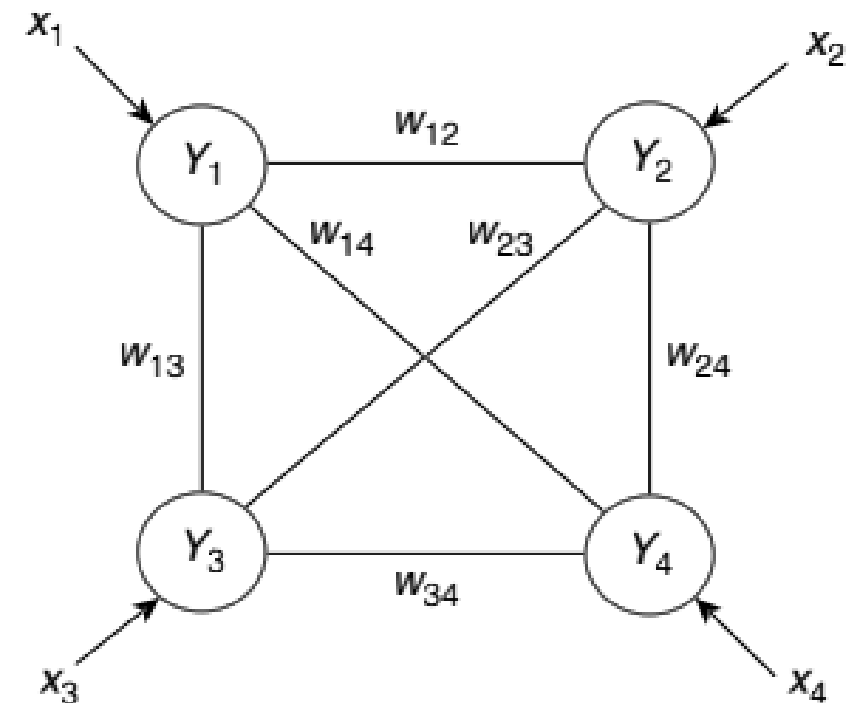
- Associative networks that do not respond immediately to an input pattern but take several steps to converge to a stable output are called iterative, or recurrent, networks. Hopfield networks, proposed by John Hopfield (1982, 1984, 1988), is one of the earliest and popular iterative auto-associative networks.

Architecture

- A Hopfield net is a fully interconnected net with symmetric weights and without any self-loop.
- This means that for a Hopfield net, $w_{ij} = w_{ji}$, for all $i \neq j$, and $w_{ii} = 0$, for all i . As the net is fully connected, the idea of various layers of processing units is no longer meaningful here. Each unit gets input from each of the remaining units, as well as the corresponding element of the input pattern. Hence, for an m -unit Hopfield net, the net input to unit Y_j is given by

$$y_{in_j} = x_j + \sum_{\substack{i=1 \\ i \neq j}}^m y_i \cdot w_{ij}$$

- (i) The net is fully connected with symmetric weights but without any self-loop.
- (ii) The net is trained with bipolar patterns so that the weight matrix is also bipolar. However, **during application binary inputs are used.**
- (iii) **Hebb** rule is used to obtain the weight matrix.
- (iv) During application, activation of a single unit Y_i is updated on the basis of the signal it receives from other units, and the input x_i to that unit.
- (v) The units update their activations in random order.
- (vi) The update of the activations of the units stops when the units reach a stable state. The stabilized set of activations of the units is taken as the output of the net.



Training

- There are several versions of Hopfield network, of which we consider the one with bipolar inputs at the time of training. Let $\mathbf{s} = [s_1, s_2, \dots, s_m]$ be an input pattern presented in bipolar form. Then the weight matrix of the Hopfield net storing the pattern $\mathbf{s}$ is obtained as:

$$w_{ij} = \begin{cases} s_i \times s_j, & \text{if } i \neq j \\ 0, & \text{if } i = j \end{cases}$$

Procedure *Hopfield-Pattern-Association*

// Given an m -unit Hopfield net with weight matrix W , to find the output of the net for the input pattern $\mathbf{x} = [x_1, x_2, \dots, x_m]$. //

Step 1. Initialize the activations of the units with the input signals.

$$y_{out_i} = x_i \quad (i = 1, \dots, m)$$

Step 2. Update the activations of the units in random order by executing Steps 3 to 4 for each unit Y_j , $j = 1, \dots, m$.

Step 3. Find the net input y_{in_j} to Y_j .

$$y_{in_j} = x_j + \sum_{i \neq j} y_{out_i} \times w_{ij}$$

Step 4. Find the activation of the unit Y_j .

$$y_{out_j} = \begin{cases} 1, & \text{if } y_{in_j} > 0 \\ y_{out_j}, & \text{if } y_{in_j} = 0 \\ 0, & \text{if } y_{in_j} < 0 \end{cases}$$

Once the activation of Y_j , i.e., y_{out_j} , is updated, it is broadcast to all other units for further updation of the activations of those units.

Step 5. Test for convergence. If there is no change in the activation of the units $Y_1, \dots, Y_m$, then the net has reached a stable state and therefore stop. Otherwise, go to Step 2.